CS 4414: HW 1.3 Report

Nolan Lizmi (nbl25) | September 29, 2025

Exact Search

Here, we want to compare my K-nearest-neighbor (KNN) algorithm from HW 1.2 to the approximate K-nearest-neighbor (AKNN) algorithm implemented using ALGLIB in HW 1.3.

For this comparison, I used K=10 (i.e., finding ten nearest neighbors) for both implementations, and ε=0 for the ALGLIB implementation (i.e., exact search). The query for both was the first entry in `data/queries_emb.json`, and the Dataset column in the below table refers to passages also contained in the `data` directory. Runtime is here broken down into total elapsed time, time for processing and parsing input, time for building the 384-dimensional tree, and time for the actual KNN search.

Algorithm	Dataset	Total Elapsed (ms)	Process & Parse (ms)	Tree Build (ms)	KNN Search (ms)
ALGLIB	passages1.json	20111.6	19177	876.91	57.5407
	passages2.json	21117.9	20027.7	1018.26	71.7311
Mine	passages1.json	5354.67	5158.08	185.226	11.3573
	passages2.json	4741.09	4522.86	206.425	11.7985

We note here that the ALGLIB implementation is slower in every single section of the algorithm. This makes intuitive sense since the ALGLIB function is adapted for use in AKNN search, whereas our algorithm is built for exact KNN. Now, we want to examine how the algorithms compare when approximation is used (ε>0).

Approximate Search

We want to see the effect on runtime and accuracy of altering the approximation constant ε in addition to the number of nearest neighbors K in the ALGLIB AKNN algorithm. I collected the following data, using the same query as before. Note that we only include the AKNN search runtime here, and note that “Accuracy” is calculated by noting the ratio of nearest neighbors present in the AKNN search that are also present in the exact KNN search.

K	ε	Dataset	Search Time (ms)	Accuracy
1	0	passages1.json	39.259	1
1	0	passages2.json	39.0847	1
1	5	passages1.json	15.7958	1

1	5	passages2.json	4.62716	1
1	10	passages1.json	4.16795	1
1	10	passages2.json	1.12298	1
1	15	passages1.json	2.23714	0
1	15	passages2.json	0.735699	1
5	0	passages1.json	41.7028	1
5	0	passages2.json	39.0492	1
5	5	passages1.json	21.6023	1
5	5	passages2.json	19.3407	1
5	10	passages1.json	4.81117	0.8
5	10	passages2.json	2.75183	0.6
5	15	passages1.json	2.61454	0.6
5	15	passages2.json	1.20401	0.6
10	0	passages1.json	57.5407	1
10	0	passages2.json	71.7311	1
10	5	passages1.json	26.1582	1
10	5	passages2.json	23.1127	1
10	10	passages1.json	4.70566	0.6
10	10	passages2.json	2.43999	0.6
10	15	passages1.json	7.6006	0.6
10	15	passages2.json	1.19162	0.5

Here, we see that the search runtimes are substantially reduced as we increase ε for all values of K. This is logical since increasing ε increases the amount of pruning that the algorithm does as it navigates the 384-dimensional tree, meaning we save time by opting to not explore more parts of the tree.

However, this improvement is balanced by a substantial reduction in accuracy for increasing ε. This is also logical since, as we prune a larger portion of the tree, we also exclude many more potential nearest neighbors from our search space.

We note that, in one instance, 50% of the “nearest neighbors” found via AKNN search are actually not true nearest neighbors (see our last data point, with Accuracy=0.5). This is a substantial portion of the returned data points, so it appears that the crux of using this ALGLIB AKNN implementation effectively is finding a good balance between runtime efficiency and accuracy. Finding this balance boils down to tuning your algorithm to a specific practical use-case, as the use-case will determine which tradeoffs can be afforded and which cannot.